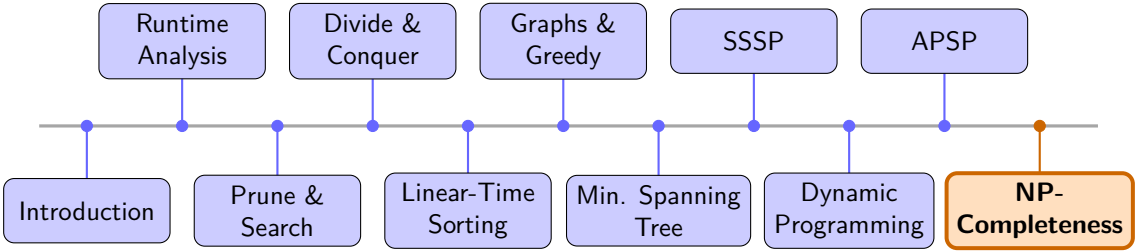
Algorithms

Dr. Mudassir Shabbir

LUMS University

April 9, 2026

Recap & What's Next



NP-Completeness

Lecture 3: SAT, 3SAT, 3COL, VC and Key Reductions

Recall: Class P

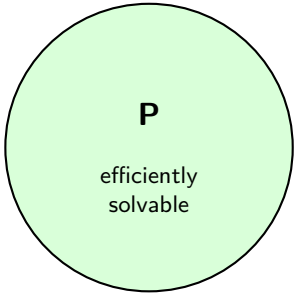
Definition

P is the class of decision problems solvable in **polynomial time**.

This means there is an algorithm with running time $O(n^k)$ for some constant k.

Examples in P:

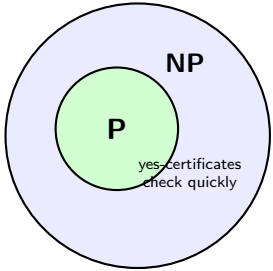
- Shortest path
- Minimum spanning tree
- Bipartite testing
- Sorting



Recall: Class NP

Definition

NP is the class of decision problems whose **Yes**-instances have certificates verifiable in **polynomial time**.



Think: a solution may be hard to find, but easy to check.

Examples in NP:

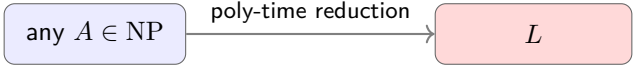
- SAT: certificate is a truth assignment
- VC: certificate is the chosen vertex set
- 3COL: certificate is the coloring

P $\subseteq$ NP because solving is at least as strong as verifying.

Recall: NP-Hard

Definition

A problem L is **NP-hard** if every problem in NP can be reduced to L in polynomial time.



Interpretation: if we could solve L efficiently, then we could solve every problem in NP efficiently.

Important

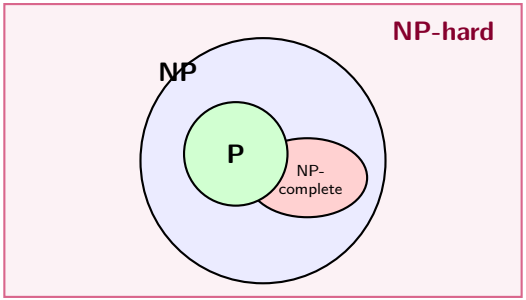
NP-hard does *not* mean the problem is in NP. It only says the problem is at least that hard.

New Definition: NP-Complete

Definition

A decision problem is **NP-complete** if:

1. it is in NP, and
2. it is NP-hard.



This lecture: we will show SAT, 3SAT, VC, and 3COL are all NP-complete.

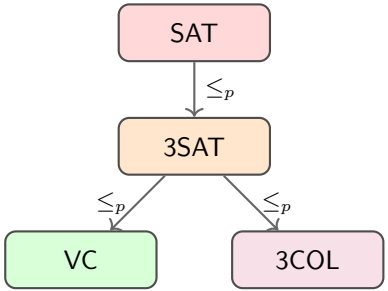
Today's Plan

Four canonical NP-complete problems:

- **SAT** — Boolean satisfiability
- **3SAT** — 3-CNF satisfiability
- **VC** — Vertex Cover (recap)
- **3COL** — Graph 3-Coloring

Three reductions we prove:

- SAT $\leq_p$ 3SAT
- 3SAT $\leq_p$ VC
- 3SAT $\leq_p$ 3COL



Boolean Satisfiability (SAT)

Definitions

A **literal** is a Boolean variable x or its negation $\neg x$.

A **clause** is an OR of literals: $(l_1 \vee l_2 \vee \cdots \vee l_k)$.

A formula in **conjunctive normal form (CNF)** is an AND of clauses.

SAT (Satisfiability)

Given a CNF formula φ, does there exist a **truth assignment** to the variables that makes φ evaluate to **true**?

Example:

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \ \wedge \ (\neg x_1 \vee x_2) \ \wedge \ (\neg x_2 \vee \neg x_3)$$

Assignment $x_1 = T,\ x_2 = T,\ x_3 = F$ ✓

SAT Example: A Clause Can Fail Even if the Formula Does Not

Consider

$$\varphi = (x_1 \vee x_2) \ \wedge \ (\neg x_1 \vee x_3) \ \wedge \ (\neg x_2 \vee \neg x_3).$$

Assignment A: $x_1 = T,\ x_2 = F,\ x_3 = T$

- $(T \vee F) = T$
- $(F \vee T) = T$
- $(T \vee F) = T$

So φ is satisfiable under Assignment A. ✓

Assignment B:
$x_1 = F,\ x_2 = T,\ x_3 = T$

- $(F \vee T) = T$
- $(T \vee T) = T$
- $(F \vee F) = F$

One false clause makes the whole CNF false. ✗

In CNF, every clause must be true. One satisfied clause helps, but one false clause kills the entire formula.

SAT Example: An Unsatisfiable Formula

Consider

$$\varphi = (x_1 \vee x_2) \ \wedge \ (x_1 \vee \neg x_2) \ \wedge \ (\neg x_1 \vee x_2) \ \wedge \ (\neg x_1 \vee \neg x_2).$$

Case 1: $x_1 = T$

- Then clauses 3 and 4 become x_2 and $\neg x_2$.
- They cannot both be true at the same time.

Case 2: $x_1 = F$

- Then clauses 1 and 2 become x_2 and $\neg x_2$.
- Again, they cannot both be true.

Checking all assignments:

x_1	x_2	φ
T	T	F
T	F	F
F	T	F
F	F	F

No assignment satisfies all clauses. ✗

An unsatisfiable SAT instance means *every* possible truth assignment fails somewhere.

SAT is NP-Complete

Cook–Levin Theorem (1971/1973)

SAT is NP-complete.

Step 1 — SAT $\in$ NP.

- Certificate: a truth assignment τ (size n).
- Verification: evaluate each clause under τ. Time $O(|\varphi|)$. ✓

Step 2 — SAT is NP-hard.

- Cook (1971) and Levin (1973) independently proved this.
- For every NP problem L and verifier V, the execution of V can be encoded as a polynomial-size SAT formula.
- This gives a poly-time reduction: $L \leq_p$ SAT for all $L \in$ NP.

SAT is the **starting point** of the NP-completeness universe.

3-SAT

Definition

A CNF formula is in **3-CNF** if every clause has **exactly 3 literals**.

3SAT: Given a 3-CNF formula φ, is it satisfiable?

Example:

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \ \wedge \ (\neg x_1 \vee x_2 \vee \neg x_3) \ \wedge \ (\neg x_1 \vee \neg x_2 \vee x_3)$$

Assignment $x_1 = T,\ x_2 = T,\ x_3 = T$:

- $(T \vee F \vee T) = T$ ✓
- $(F \vee T \vee F) = T$ ✓
- $(F \vee F \vee T) = T$ ✓

Key fact

3SAT $\leq_p$ SAT trivially (3-CNF is CNF). The hard direction is SAT $\leq_p$ 3SAT.

3SAT Example: Exactly Three Literal Slots Per Clause

Valid 3-CNF instance:

$$(x_1 \vee \neg x_2 \vee x_3) \ \wedge \ (\neg x_1 \vee x_2 \vee x_4)$$

Each clause has exactly three literal positions, so this is a 3SAT instance.

Assignment:

$x_1 = T,\ x_2 = F,\ x_3 = F,\ x_4 = T$

- $(T \vee T \vee F) = T$
- $(F \vee F \vee T) = T$

Not a 3SAT instance:

$$(x_1 \vee x_2) \ \wedge \ (\neg x_2 \vee x_3 \vee x_4 \vee x_5)$$

This is still SAT, but not 3SAT, because one clause has 2 literals and another has 4.

3SAT is not about formulas with *at most* 3 literals per clause here; in this lecture we use the exact-3-literal version.

Reduction: SAT $\leq_p$ 3SAT

Goal: Convert any CNF formula into an *equivalent* 3-CNF formula.

Given clause $C = (l_1 \vee l_2 \vee \cdots \vee l_k)$, transform it case by case:

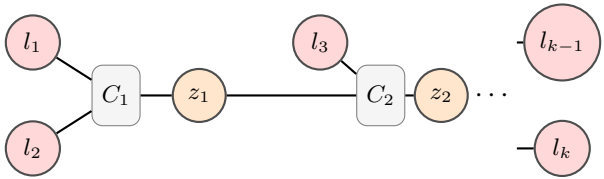
Case	Original clause	Replacement (fresh variables z_i)
$k = 1$	(l_1)	$(l_1 \vee z \vee z') \wedge (l_1 \vee z \vee \neg z') \wedge (l_1 \vee \neg z \vee z') \wedge (l_1 \vee \neg z \vee \neg z')$
$k = 2$	$(l_1 \vee l_2)$	$(l_1 \vee l_2 \vee z) \wedge (l_1 \vee l_2 \vee \neg z)$
$k = 3$	$(l_1 \vee l_2 \vee l_3)$	unchanged
$k > 3$	$(l_1 \vee \cdots \vee l_k)$	(see next slide)

Running time: Each clause of length k produces $O(k)$ new 3-clauses and $O(k)$ fresh variables. ✓

SAT $\leq_p$ 3SAT: The Long-Clause Case ($k > 3$)

For a clause $(l_1 \vee l_2 \vee \cdots \vee l_k)$ with $k > 3$:
Introduce fresh variables $z_1, z_2, \ldots, z_{k-3}$ and replace with the $k-2$ clauses:

$$\underbrace{(l_1 \vee l_2 \vee z_1)}_{\text{clause } 1} \wedge \underbrace{(\neg z_1 \vee l_3 \vee z_2)}_{\text{clause } 2} \wedge \cdots \wedge \underbrace{(\neg z_{k-4} \vee l_{k-2} \vee z_{k-3})}_{\text{clause } k-3} \wedge \underbrace{(\neg z_{k-3} \vee l_{k-1} \vee l_k)}_{\text{clause } k-2}$$



Correctness: If the original clause is satisfied by some $l_i = T$, set $z_1 = \cdots = z_{i-2} = T$ and $z_{i-1} = \cdots = z_{k-3} = F$. Every new clause becomes satisfied. Conversely, if the original clause is violated, all $l_i = F$ forces all z_j to satisfy contradictory requirements. ✓

3SAT is NP-Complete

Theorem

3SAT is NP-complete.

Step 1 — 3SAT $\in$ NP. Certificate: truth assignment. Verify each 3-clause in $O(1)$. ✓

Step 2 — 3SAT is NP-hard. We just showed SAT $\leq_p$ 3SAT. Since SAT is NP-hard:

$$\forall L \in \text{NP}: \quad L \leq_p \text{SAT} \leq_p \text{3SAT}$$

By transitivity of reductions, $L \leq_p$ 3SAT for every $L \in$ NP. ✓

How about 2SAT?

Vertex Cover (Recap)

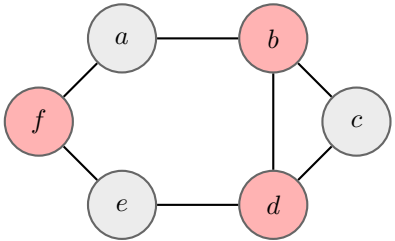
Definition

Given a graph $G = (V, E)$ and integer k.

VC: Does there exist a set $S \subseteq V$ with $|S| \leq k$ such that every edge has at least one endpoint in S?

Example ($k = 3$): ✓Yes

Red vertices $\{b, d, f\}$ cover every edge.

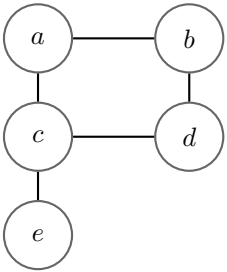


Certificate for VC $\in$ NP:
The set S itself. Check each edge has an endpoint in S: $O(|E|)$. ✓

Vertex Cover Example: What Must Be Chosen?

Let G have edges

$$E = \{(a, b), (a, c), (b, d), (c, d), (c, e)\}.$$



Question 1: Is there a vertex cover of size 2?

Yes: choose $S = \{a, d\}$? ✗ edge (c, e) is uncovered.

Choose $S = \{b, c\}$? ✓

- (a, b) covered by b
- (a, c) covered by c
- (b, d) covered by b
- (c, d) covered by c
- (c, e) covered by c